



UNIVERSIDADE ESTADUAL DE SANTA CRUZ

FLÁVIA ALESSANDRA SANTOS DE JESUS

**DESENVOLVIMENTO DE UM SISTEMA WEB MONTADOR
DE BAREMA DE ATIVIDADES COMPLEMENTARES PARA
O CURSO DE CIÊNCIA DA COMPUTAÇÃO DA UESC**

**ILHÉUS - BAHIA
2026**

FLÁVIA ALESSANDRA SANTOS DE JESUS

**DESENVOLVIMENTO DE UM SISTEMA WEB MONTADOR
DE BAREMA DE ATIVIDADES COMPLEMENTARES PARA
O CURSO DE CIÊNCIA DA COMPUTAÇÃO DA UESC**

Trabalho de Conclusão de Curso submetido à Universidade Estadual de Santa Cruz, como requisito para obtenção do grau de Bacharel em CIÊNCIA DA COMPUTAÇÃO.

Orientador: Álvaro Vinícius de Souza Coelho.

**ILHÉUS - BAHIA
2026**

FLÁVIA ALESSANDRA SANTOS DE JESUS

**DESENVOLVIMENTO DE UM SISTEMA WEB MONTADOR
DE BAREMA DE ATIVIDADES COMPLEMENTARES PARA
O CURSO DE CIÊNCIA DA COMPUTAÇÃO DA UESC**

**Trabalho de Conclusão de Curso subme-
tido à Universidade Estadual de Santa
Cruz como requisito para obtenção do
grau de Bacharel em CIÊNCIA DA
COMPUTAÇÃO.**

Ilhéus, Julho de 2026.

Orientador(a): Álvaro Vinícius de Souza
Coelho
Universidade Estadual de Santa Cruz

Prof^a. Martha Ximena Torres Delgado
Universidade Estadual de Santa Cruz

Prof.
Universidade Estadual de Santa Cruz

Agradecimentos

Resumo

Palavras-chave: .

Abstract

Keywords: .

Lista de ilustrações

Figura 1 – Fluxo de desenvolvimento adotado na construção do protótipo.	30
Figura 2 – Diagrama de Casos de Uso do sistema Barema.	34
Figura 3 – Diagrama de Classes do núcleo de domínio do sistema.	37
Figura 4 – Modelo Entidade-Relacionamento (MER) do banco de dados.	38
Figura 5 – Tela inicial do sistema Barema.	39
Figura 6 – Formulário do barema com atividades carregadas.	39
Figura 7 – Tela de <i>login</i> administrativo.	41
Figura 8 – Painel do coordenador para análise das solicitações.	42

Lista de abreviaturas e siglas

Sumário

1	INTRODUÇÃO	13
1.1	Objetivos	14
1.1.1	Objetivo Geral	14
1.1.2	Objetivos Específicos	14
1.2	Contribuições do Trabalho	15
1.3	Organização do Documento	15
2	PROBLEMA	16
2.1	Definição	16
2.2	Desafios	17
2.2.1	Conformidade com as Regras de Negócio	17
2.2.2	Organização e Indexação dos Documentos	18
2.2.3	Extração e Pré-validação de Dados dos Certificados	18
3	JUSTIFICATIVA	20
3.1	Âmbito Administrativo	20
3.2	Meio Acadêmico	20
3.3	Justificativa Técnica	21
4	FUNDAMENTAÇÃO TEÓRICA	22
4.1	Conceitos Fundamentais	22
4.1.1	Arquitetura em Camadas e o Padrão MVC	22
4.1.2	Mapeamento Objeto-Relacional e Persistência de Dados	22
4.1.3	Processamento de Documentos Digitais e Extração de Texto	23
4.1.4	Reatividade de Interface e Tempo de Resposta	23
4.2	Tecnologias de Apoio à Implementação	24
4.2.1	Linguagem Python e Microframework Flask	24
4.2.2	Processamento de PDF e Reconhecimento Óptico de Caracteres	24
4.2.3	Persistência de Dados com SQLite e SQLAlchemy	25
4.2.4	Comunicação Assíncrona e Reatividade da Interface	25
4.2.5	Serviços de Notificação	25
5	TRABALHOS CORRELATOS	26
5.1	Automação de Processos Acadêmico-Administrativos	26
5.2	Sistemas Web para Gerenciamento de Atividades Complementares	26
5.3	Extração Automatizada de Dados em Documentos PDF	27

5.4	Síntese Comparativa	27
6	METODOLOGIA	29
6.1	Caracterização do Trabalho	29
6.2	Procedimentos Metodológicos	29
6.2.1	Prototipagem Evolutiva	29
6.2.2	Levantamento de Requisitos	30
6.2.3	Definição da Arquitetura, Ferramentas e Implementação	31
6.3	Estratégia de Avaliação	31
6.3.1	CrITÉrios de Sucesso do Protótipo	32
7	SOLUÇÃO PROPOSTA	34
7.1	Visão Geral e Casos de Uso	34
7.2	Requisitos do Sistema	35
7.3	Arquitetura e Persistência de Dados no Projeto	36
7.4	Implementação	38
7.4.1	Montagem do Barema	38
7.4.2	Extração de Dados	40
7.4.3	Pré-validação e Geração do Documento	40
7.4.4	Painel de Análise (Coordenação)	41
8	RESULTADOS E DISCUSSÃO	43
8.1	Resultados Obtidos	43
8.2	Validação do Protótipo	44
8.3	Discussão da Solução	44
8.4	Limitações Observadas	45
9	CONCLUSÃO	46
	REFERÊNCIAS	48

1 Introdução

A formação em nível de graduação transcende às atividades realizadas em sala de aula. No campo da Computação, as Diretrizes Curriculares Nacionais reconhecem a importância de experiências complementares para ampliar a formação do discente, estimulando vivências práticas, independentes e interdisciplinares, especialmente em articulação com o mundo do trabalho (Brasil 2016). No curso de Ciência da Computação da Universidade Estadual de Santa Cruz (UESC), essas experiências são consolidadas por meio das Atividades Complementares (AC), requisito obrigatório para a integralização curricular.

O aproveitamento dessas atividades depende da comprovação documental das horas realizadas pelo discente. Esse processo se relaciona também às normas nacionais sobre carga horária mínima e integralização dos cursos de graduação, que estabelecem parâmetros para a organização dos percursos formativos em cursos presenciais (Brasil 2007). No âmbito do Colegiado de Ciência da Computação (COLCIC), a validação das AC exige que o estudante apresente um Barema organizado, contendo a relação das atividades, a carga horária correspondente e os documentos comprobatórios reunidos em um único arquivo.

Embora esse procedimento seja essencial para a regularidade acadêmica do discente, sua execução ainda ocorre de forma predominantemente manual. O estudante precisa reunir certificados e comprovantes dispersos, organizar os arquivos em formato PDF, preencher a tabela de atividades, calcular as cargas horárias por categoria, observar os tetos previstos no regulamento e indicar corretamente a paginação dos comprovantes no documento final. Pequenas alterações na ordem dos arquivos ou na quantidade de certificados anexados podem comprometer toda a indexação já realizada, exigindo retrabalho na montagem do Barema.

Esse fluxo manual produz três dificuldades principais. A primeira está relacionada à complexidade regulatória, pois o discente precisa interpretar e aplicar corretamente limites de carga horária por categoria. A segunda diz respeito à organização documental, uma vez que a paginação e a unificação dos comprovantes são suscetíveis a erros quando realizadas sem apoio automatizado. A terceira envolve o retrabalho administrativo da coordenação, que precisa devolver e reavaliar submissões com falhas formais antes mesmo de analisar o mérito acadêmico das atividades apresentadas.

Nesse contexto, a automação de processos pode contribuir para reduzir erros decorrentes de tarefas repetitivas, diminuindo a intervenção manual em etapas operacionais e aumentando a eficiência do fluxo de trabalho (Abidemi 2024). Entretanto, ferramentas genéricas de manipulação de PDF não resolvem integralmente o problema, pois permitem

apenas unir ou reorganizar documentos, sem compreender as regras específicas do Barema do COLCIC, os limites de carga horária ou a necessidade de indexação coerente dos comprovantes.

Diante desse cenário, este trabalho propõe o desenvolvimento de um sistema *web* de apoio à preparação do Barema de Atividades Complementares do curso de Ciência da Computação da UESC. A solução não busca substituir a análise institucional realizada pelo colegiado, mas atuar como uma camada de apoio à organização, à pré-validação e à geração padronizada do documento. O objetivo é reduzir erros formais antes da submissão oficial, melhorar a consistência das informações apresentadas e apoiar tanto o discente quanto a coordenação no fluxo de aproveitamento das Atividades Complementares.

1.1 Objetivos

Para orientar o desenvolvimento da solução proposta, o trabalho foi estruturado em torno dos seguintes objetivos.

1.1.1 Objetivo Geral

Desenvolver um protótipo funcional de sistema *web* para apoiar a automação, a pré-validação e a geração padronizada de Baresmas de Atividades Complementares voltado aos discentes e à coordenação do COLCIC/UESC.

1.1.2 Objetivos Específicos

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

1. Realizar o levantamento de requisitos funcionais, normativos e operacionais relacionados ao processo de preparação e submissão do Barema.
2. Desenvolver uma aplicação *web* capaz de organizar dados do discente, certificados e atividades em um fluxo único de preenchimento.
3. Implementar mecanismos de extração de dados, aplicação de regras de negócio e geração de um arquivo PDF unificado, indexado e paginado.
4. Estruturar a persistência das solicitações de análise, favorecendo a rastreabilidade do processo e o envio de *feedbacks*.
5. Avaliar o funcionamento do protótipo quanto à conformidade dos cálculos de horas, à detecção de inconsistências e à aplicação dos limites regulamentares do curso.

1.2 Contribuições do Trabalho

Como contribuição para a área de Engenharia de Software aplicada à gestão acadêmica, este trabalho apresenta:

- um protótipo funcional voltado à preparação automatizada do Barema de Atividades Complementares do COLCIC/UESC;
- um fluxo integrado para preenchimento de dados, anexação de certificados, organização de comprovantes e geração de PDF unificado;
- um mecanismo de pré-validação que confronta informações declaradas pelo discente com dados extraídos dos documentos anexados;
- uma modelagem de regras de negócio capaz de representar limites de carga horária e categorias previstas no regulamento;
- um painel administrativo de apoio à coordenação para acompanhamento das solicitações submetidas.

1.3 Organização do Documento

Este documento está estruturado em capítulos organizados da seguinte forma. O Capítulo 2 apresenta o detalhamento do problema e os desafios do processo atual. O Capítulo 3 expõe a justificativa administrativa, acadêmica e técnica que motiva o projeto. O Capítulo 4 apresenta a fundamentação teórica, abordando os conceitos e as tecnologias que embasam o desenvolvimento da ferramenta. O Capítulo 5 discute os trabalhos correlatos, comparando a abordagem deste projeto com outras soluções existentes. O Capítulo 6 descreve a metodologia, o levantamento de requisitos e as estratégias adotadas para a construção do protótipo. O Capítulo 7 detalha a arquitetura e a engenharia da solução técnica implementada. O Capítulo 8 apresenta a avaliação dos cenários de teste, discutindo a aplicabilidade e o atendimento aos critérios de sucesso. O Capítulo 9 sintetiza as conclusões obtidas e indica diretrizes para trabalhos futuros. Por fim, as referências bibliográficas complementam o documento, fornecendo o suporte teórico para o desenvolvimento.

2 Problema

Este capítulo aborda a conceituação do problema para a solução proposta, bem como seus desafios.

2.1 Definição

As Atividades Complementares (AC) constituem um requisito obrigatório para a integralização curricular no curso de Ciência da Computação da UESC. Seu cumprimento envolve a participação do discente em eventos científicos, estágios, projetos de extensão, monitorias e outras vivências formativas. Para que essas horas sejam reconhecidas oficialmente pelo Colegiado do Curso (COLCIC), o aluno deve organizar e submeter o Barema: um documento unificado que reúne os comprovantes das atividades realizadas, categoriza-os segundo as normas vigentes e totaliza a carga horária acumulada por categoria. A submissão oficial deste documento é feita via Peticionamento Eletrônico, para que a coordenação do curso avalie o barema e dê um parecer sobre o aproveitamento das horas solicitadas pelo discente.

O processo de elaboração desse documento, contudo, é inteiramente manual e descentralizado. Não existe ferramenta oficial fornecida pela instituição para auxiliar nessa tarefa. O discente precisa reunir individualmente cada certificado em formato PDF, organizá-los em um único arquivo, preencher manualmente uma tabela de indexação — indicando em qual página do documento unificado cada comprovante se encontra — e calcular as cargas horárias respeitando os tetos máximos permitidos por categoria, conforme as regras específicas do curso de bacharelado em Ciência da Computação da UESC. Cada uma dessas etapas é executada de forma independente, sem qualquer validação automatizada, e qualquer alteração na ordem ou no conjunto de documentos pode gerar um efeito cascata de erros que inviabiliza o aproveitamento da indexação previamente realizada.

Esse cenário resulta em três problemas concretos e recorrentes. O primeiro é a complexidade regulatória: os alunos encontram dificuldade em aplicar corretamente as regras de carga horária definidas pelo COLCIC — como o limite de horas permitido por categoria de atividade —, o que leva a cálculos errôneos e à inclusão indevida de horas que excedem os tetos regulamentares. O segundo é a propensão a erros de indexação: o preenchimento manual da coluna de páginas é suscetível a falhas simples, porém de alto impacto — um único certificado inserido ou removido do conjunto invalida todos os números de página subsequentes, obrigando o discente a reiniciar o processo de edição, reordenação e reescrita da tabela integralmente. Relatos colhidos junto a discentes evidenciam casos

em que o documento precisou ser refeito três vezes consecutivas por conta exclusivamente desse tipo de inconsistência. O terceiro problema é o retrabalho administrativo gerado na coordenação: o envio de baremas incorretos — seja por erros de cálculo, por indexação equivocada ou por separação indevida entre o barema e os documentos comprobatórios — obriga a coordenação a realizar múltiplas rodadas de revisão com cada aluno, atrasando o aproveitamento das atividades e, conseqüentemente, o processo de colação de grau.

É importante destacar que as soluções genéricas de manipulação de PDF disponíveis no mercado, como o (iLovePDF 2026), não resolvem esse problema. De acordo com sua documentação oficial, essa ferramenta opera sobre a estrutura do arquivo, permitindo unir, dividir ou comprimir documentos. Embora atualmente ela ofereça recursos para resumir ou comparar textos, sua lógica é de propósito geral, desconhecendo as regras específicas para a montagem do Barema de AC, ou outro contexto específico. Essa limitação evidencia que tais ferramentas funcionam apenas como camadas tecnológicas isoladas, falhando em atuar como um Sistema de Informação eficaz que, segundo (Laudon e Laudon 2014), deve compreender a organização e as regras que o cercam para atingir seus objetivos. No caso do COLCIC, essas ferramentas permitem unir arquivos PDF, mas não calculam cargas horárias, não aplicam tetos regulamentares e não geram a tabela de indexação automaticamente. A responsabilidade sobre toda a lógica de negócio permanece, portanto, integralmente com o aluno. O problema não é a ausência de uma ferramenta de junção de arquivos — é a ausência de um sistema que compreenda as regras do processo e as aplique de forma integrada à manipulação dos documentos.

2.2 Desafios

O desenvolvimento de um sistema de apoio à elaboração do Barema do COLCIC envolve desafios que ultrapassam a simples junção de arquivos PDF. A solução precisa articular regras acadêmicas, documentos digitais heterogêneos e uma experiência de uso suficientemente clara para reduzir erros antes da submissão oficial ao colegiado. Nesse sentido, os principais desafios concentram-se na tradução das normas do barema para regras computacionais, na organização confiável dos documentos anexados e na extração preliminar de informações dos certificados.

2.2.1 Conformidade com as Regras de Negócio

Um dos principais desafios consiste em representar, no sistema, as regras de negócio definidas para o aproveitamento das Atividades Complementares. Essas regras envolvem categorias de atividades, limites máximos de carga horária, condições mínimas de aproveitamento e critérios específicos de organização dos comprovantes. A dificuldade não está apenas em somar horas, mas em assegurar que o cálculo respeite os tetos

regulamentares estabelecidos para cada tipo de atividade.

Por exemplo, quando o discente possui certificados que excedem o limite permitido em uma determinada categoria, o sistema deve considerar apenas a carga horária aproveitável segundo o regulamento. Assim, a ferramenta precisa diferenciar a carga horária apresentada nos certificados da carga horária efetivamente computável para integralização curricular. Essa distinção é essencial para reduzir erros de preenchimento e evitar que o documento final apresente valores incompatíveis com as normas do curso.

As regras de negócio e o fluxo de validação serão representados em diagramas próprios da solução, permitindo visualizar como as categorias, os limites e as verificações são articulados no processo de geração do barema. O mapeamento dessas regras será apresentado na Seção ??, que reunirá os diagramas utilizados para documentar a lógica de funcionamento da ferramenta.

2.2.2 Organização e Indexação dos Documentos

Outro desafio relevante está relacionado à organização dos documentos comprobatórios. No processo manual, o discente precisa reunir certificados de diferentes origens, ordenar os arquivos, consolidá-los em um único PDF e informar corretamente a página em que cada comprovante se encontra. Qualquer alteração posterior na ordem dos documentos pode comprometer a indexação já preenchida, exigindo a revisão de toda a tabela.

No sistema proposto, esse desafio é tratado por meio de uma organização assistida dos arquivos anexados. A ferramenta deve manter a relação entre categoria, ordem do certificado, quantidade de páginas e posição do documento no arquivo final. Com isso, a indexação deixa de depender exclusivamente da conferência manual do aluno e passa a ser derivada do próprio fluxo de montagem do barema.

Essa organização também precisa considerar que o sistema opera sem exigir uma conta persistente para o discente. Portanto, durante a montagem do documento, os dados manipulados devem ser tratados como informações temporárias de trabalho, suficientes para gerar o arquivo final e apoiar a submissão, mas sem transformar cada tentativa incompleta em um registro permanente no banco de dados.

2.2.3 Extração e Pré-validação de Dados dos Certificados

A extração de informações dos certificados representa outro ponto de complexidade. Os documentos enviados pelos alunos podem possuir formatos, layouts e padrões textuais diferentes, o que dificulta a identificação automática de dados como nome do participante, carga horária e data da atividade. Mesmo quando os certificados possuem camada textual acessível, a forma como essas informações aparecem pode variar entre instituições emissoras.

Essa dificuldade torna-se ainda mais evidente quando o arquivo, embora esteja no formato PDF, não possui texto pesquisável em sua estrutura interna. Em muitos casos, o certificado é composto por uma imagem digitalizada ou por uma captura inserida dentro de um PDF, o que impede a leitura direta das informações por mecanismos tradicionais de extração textual. Nesses cenários, a identificação automática de dados depende de recursos complementares, como o Reconhecimento Óptico de Caracteres (OCR), e pode sofrer variações de acordo com a resolução da imagem, a qualidade da digitalização e a disposição visual dos elementos no certificado.

Diante disso, a pré-validação automatizada deve ser compreendida como um mecanismo de apoio, e não como substituição da análise institucional. O sistema pode confrontar informações declaradas pelo discente com dados identificados no arquivo anexado, sinalizando possíveis divergências, como diferença de carga horária, inconsistência de nome ou duplicidade de documentos. Essas sinalizações contribuem para que erros formais sejam percebidos antes da submissão oficial.

Ainda assim, a decisão final sobre o aproveitamento das atividades permanece sob responsabilidade do colegiado. O papel da ferramenta é organizar o material, aplicar verificações objetivas e produzir um documento mais coerente para análise, reduzindo retrabalho tanto para o discente quanto para a coordenação.

3 Justificativa

A transição para um modelo computacional especializado não visa apenas a digitalização de documentos, mas a garantia da integridade dos dados e da otimização de processos por meio da Engenharia de Software, que consiste na aplicação de uma abordagem sistemática e disciplinada para o desenvolvimento de soluções confiáveis (Pressman e Maxim 2016). A necessidade deste sistema fundamenta-se na aplicação de soluções automatizadas como resposta a processos manuais ineficientes, estruturando-se em duas frentes: a otimização administrativa de recursos e a mitigação de riscos acadêmicos para o estudante.

3.1 Âmbito Administrativo

Do ponto de vista administrativo, a coordenação do curso opera com recursos humanos limitados e precisa analisar individualmente cada barema submetido. O volume de documentos incorretos que retornam para correção representa um custo operacional real: cada revisão exige nova comunicação com o discente, novo prazo de reenvio e nova análise pelo coordenador.

Nesse cenário, a automação do fluxo não é apenas uma conveniência, mas uma aplicação direta de princípios consolidados na Engenharia de Software. A automação minimiza a intervenção humana em tarefas repetitivas, reduzindo o risco de erros operacionais e aumentando a eficiência (Abidemi 2024). Assim, a indexação de páginas e o cálculo de cargas horárias são tarefas determinísticas e com regras bem definidas, beneficiando-se diretamente dessa abordagem para otimizar a capacidade de atendimento do colegiado.

3.2 Meio Acadêmico

Do ponto de vista acadêmico, o processo atual impõe ao discente um ônus desproporcional à natureza da tarefa. Montar um barema não é uma atividade de aprendizado — é uma tarefa burocrática de baixa complexidade cognitiva, mas de alto potencial de erro quando executada manualmente. Exigir que o aluno dedique tempo e atenção a contar páginas e somar horas em planilhas é desviar energia que poderia ser investida nas próprias atividades complementares que o processo visa reconhecer. Além disso, erros nessa etapa têm consequências concretas: atrasos no aproveitamento das AC podem comprometer o cumprimento dos prazos para a colação de grau, impactando diretamente o planejamento acadêmico e profissional do estudante.

3.3 Justificativa Técnica

Por fim, a justificativa técnica do projeto reside no fato de que a solução proposta não é apenas um agregador de arquivos, mas um sistema que codifica as regras de negócio do COLCIC. Isso significa que as validações de carga horária máxima por categoria, a geração automática da tabela de indexação e a unificação dos comprovantes ocorrem de forma integrada e transparente para o usuário. O resultado é um documento gerado com garantia de conformidade regulatória — algo que nenhuma ferramenta genérica hoje disponível é capaz de oferecer para esse contexto específico.

4 Fundamentação Teórica

Este capítulo apresenta a base conceitual que sustenta o desenvolvimento e a arquitetura do sistema de apoio ao Barema do COLCIC. São abordados os princípios de arquitetura de sistemas para a Web, os conceitos de processamento de documentos binários e mineração de texto, além dos fundamentos de *interfaces* reativas e persistência de dados.

4.1 Conceitos Fundamentais

4.1.1 Arquitetura em Camadas e o Padrão MVC

O desenvolvimento de aplicações web modernas exige a adoção de modelos arquiteturais que favoreçam a modularidade e o isolamento de responsabilidades. Tradicionalmente, o padrão arquitetural em camadas organiza o sistema em níveis distintos — apresentação, negócio e persistência —, de forma que alterações em uma camada não comprometam as demais. Dentro desse modelo, o padrão *Model-View-Controller* (MVC) é amplamente empregado para separar a lógica de apresentação da lógica de negócio e de acesso a dados (Pressman e Maxim 2016).

Fowler 2006 descreve o MVC como um dos padrões arquiteturais mais relevantes para aplicações web, destacando que a separação entre o modelo de dados, a camada de visualização e o controlador de fluxo é o que permite a evolução independente de cada componente sem ruptura do sistema como um todo. No contexto deste trabalho, essa separação é crítica: as regras de negócio do COLCIC precisam estar isoladas da interface e da camada de geração de documentos, de modo que alterações no regulamento possam ser incorporadas sem reescrever o sistema inteiro.

4.1.2 Mapeamento Objeto-Relacional e Persistência de Dados

A camada de persistência de dados em sistemas orientados a objetos frequentemente enfrenta o problema da incompatibilidade de impedância, caracterizado pela diferença entre o modelo conceitual de objetos na memória e o modelo relacional das tabelas em um banco de dados SQL. Para mitigar esse problema, utiliza-se o Mapeamento Objeto-Relacional (*Object-Relational Mapping* — ORM).

O ORM atua como uma camada de abstração que traduz de forma transparente as operações sobre objetos computacionais em comandos de manipulação de dados em bancos relacionais (Fowler 2006). Essa abstração permite que a aplicação opere sobre

estruturas nativas da linguagem — como dicionários e listas de objetos em Python — e delegue ao ORM a responsabilidade de sincronizar essas estruturas com o banco de dados, minimizando o acoplamento direto com o motor de persistência escolhido.

4.1.3 Processamento de Documentos Digitais e Extração de Texto

A manipulação de arquivos no formato PDF (*Portable Document Format*) introduz desafios relacionados à natureza não estruturada das informações. Diferente de linguagens de marcação estruturadas (como XML ou JSON), o PDF foca na fidelidade de renderização visual e na disposição física de caracteres em coordenadas bidimensionais, omitindo frequentemente tags de estrutura semântica ou separadores lógicos de parágrafos.

Para transformar esses fluxos de bytes brutos em dados inteligíveis pela lógica de negócio, utilizam-se bibliotecas especializadas na decodificação da estrutura interna do arquivo. A partir do conteúdo textual extraído, aplicam-se técnicas de processamento baseadas em Expressões Regulares (*Regular Expressions* — Regex). As expressões regulares operam como autômatos finitos determinísticos que realizam o casamento de padrões textuais (HOPCROFT, MOTWANI e ULLMAN 2006), permitindo varrer o conteúdo do documento em busca de tokens numéricos adjacentes a qualificadores de carga horária — como "horas", "h" e "carga horária" — para identificar automaticamente os metadados operacionais dos certificados.

4.1.4 Reatividade de Interface e Tempo de Resposta

A experiência do usuário em plataformas utilitárias é diretamente influenciada pela latência observada nas atualizações da interface. Nielsen estabelece três limiares fundamentais de tempo de resposta que permanecem relevantes desde suas formulações originais: respostas em até 0,1 segundo sustentam a percepção de instantaneidade; respostas em até 1,0 segundo mantêm o fluxo de pensamento do usuário sem interrupções perceptíveis; e respostas que ultrapassam 10 segundos comprometem a atenção do usuário e exigem *feedback* visual explícito de progresso (Nielsen 1993).

Para alcançar esse patamar de reatividade em operações de manipulação de arquivos, adota-se o modelo de computação assíncrona na camada de apresentação. Utilizando *APIs* nativas dos navegadores modernos, como a *Fetch API*, a interface web envia dados e arquivos binários em segundo plano para as rotas do servidor. O servidor processa as informações e devolve respostas leves em formato JSON, permitindo que o estado visual do navegador seja modificado dinamicamente via JavaScript sem a necessidade de recarregamento completo da página e sem interromper o fluxo de pensamento do usuário.

4.2 Tecnologias de Apoio à Implementação

A implementação do sistema exigiu a combinação de tecnologias voltadas à construção de aplicações web, ao processamento de documentos digitais, à persistência de dados e à comunicação assíncrona entre interface e servidor. A escolha dessas tecnologias foi orientada pelas necessidades do domínio: manipular arquivos enviados pelo discente, aplicar regras de negócio, gerar documentos consolidados e registrar solicitações de análise.

4.2.1 Linguagem Python e Microframework Flask

Em sistemas web que concentram regras de negócio específicas e operações de processamento sob demanda, a escolha da linguagem e do *framework* influencia diretamente a organização da aplicação. O Python foi adotado por oferecer um ecossistema amplo para manipulação de arquivos, processamento textual e integração com bibliotecas externas. O Flask, por sua vez, caracteriza-se como um *microframework* que fornece recursos essenciais para roteamento HTTP sem impor uma estrutura rígida de projeto (Grinberg 2018).

No contexto deste trabalho, o Flask atua como camada de controle da aplicação, recebendo as requisições do discente, coordenando o envio de certificados, acionando os serviços de extração e validação e encaminhando os dados necessários para a geração do documento final. Essa organização favorece a separação entre interface, regras de negócio e persistência, mantendo coerência com a arquitetura em camadas discutida anteriormente.

4.2.2 Processamento de PDF e Reconhecimento Óptico de Caracteres

O processamento de certificados em formato digital exige ferramentas capazes de acessar o conteúdo interno de documentos PDF e, quando necessário, lidar com arquivos sem camada textual. O PyMuPDF fornece acesso programático a documentos PDF, permitindo extrair texto, inspecionar páginas e manipular arquivos de forma automatizada (Artifex Software, Inc. 2026). Para documentos digitalizados como imagem, o Tesseract oferece recursos de Reconhecimento Óptico de Caracteres (OCR), convertendo conteúdo visual em texto processável (Google 2026).

Neste trabalho, essas tecnologias apoiam a extração de informações relevantes dos certificados, como carga horária, datas e dados textuais associados ao discente. O PyMuPDF é utilizado como via principal quando o documento possui texto nativo, enquanto o OCR funciona como alternativa para certificados digitalizados ou imagens. Essa combinação amplia a capacidade do sistema de lidar com documentos heterogêneos, embora a qualidade da extração continue dependente da legibilidade e da estrutura dos arquivos enviados.

4.2.3 Persistência de Dados com SQLite e SQLAlchemy

A persistência de dados é necessária para registrar solicitações de análise, usuários administrativos e informações relacionadas ao acompanhamento dos baremas submetidos. O SQLite foi adotado por ser um banco relacional embutido, adequado a protótipos e aplicações de menor complexidade operacional. O SQLAlchemy, por sua vez, oferece uma camada de mapeamento objeto-relacional que permite representar entidades da aplicação como objetos, reduzindo o acoplamento direto com comandos SQL (SQLAlchemy Authors 2026).

No sistema desenvolvido, essa combinação permite registrar o histórico de solicitações e o estado de análise dos baremas, preservando a separação entre o modelo de domínio e a infraestrutura de banco de dados. Assim, entidades como solicitações, usuários e dados de contato podem ser manipuladas pela aplicação sem que a lógica de negócio dependa diretamente da estrutura física das tabelas.

4.2.4 Comunicação Assíncrona e Reatividade da Interface

A interação do discente com o sistema envolve o envio de arquivos, atualização de cargas horárias e exibição de mensagens de validação. Para evitar recarregamentos completos da página a cada operação, utiliza-se JavaScript em conjunto com a *Fetch API*, que permite a realização de requisições HTTP assíncronas a partir do navegador.

Essa estratégia contribui para uma experiência mais fluida, pois a interface pode enviar certificados ao servidor, receber respostas em formato estruturado e atualizar apenas os elementos necessários da página. No contexto do Barema, isso é especialmente importante porque o discente precisa perceber rapidamente os efeitos de suas ações, como anexação de comprovantes, alertas de divergência e atualização dos totais de carga horária.

4.2.5 Serviços de Notificação

Além da montagem e submissão do Barema, o sistema prevê apoio ao retorno da análise realizada pela coordenação. Para esse fim, serviços externos de comunicação podem ser utilizados para envio de mensagens ao discente. A plataforma Twilio oferece APIs para integração com diferentes canais de comunicação, incluindo mensageria e serviços associados a envio de mensagens eletrônicas (Twilio Inc. 2026).

Neste trabalho, esse recurso é tratado como apoio ao fluxo administrativo de retorno ao discente. Sua adoção permite automatizar parte da comunicação após o registro do parecer, mas não substitui a responsabilidade institucional da coordenação sobre a análise e a decisão final.

5 Trabalhos Correlatos

Este capítulo apresenta trabalhos relacionados ao sistema proposto, organizados em torno dos três eixos temáticos que fundamentam esta pesquisa: a automação de processos acadêmico-administrativos em instituições de ensino superior, o desenvolvimento de sistemas web voltados ao gerenciamento de atividades complementares, e a extração automatizada de dados a partir de documentos em formato PDF.

5.1 Automação de Processos Acadêmico-Administrativos

Fernandes 2021 apresentou, no âmbito da Universidade Federal de Santa Catarina, a aplicação de Automação Robótica de Processos (RPA) para o Planejamento e Acompanhamento de Atividades Docentes (PAAD). O trabalho parte da premissa de que processos repetitivos e de baixo cunho intelectual são os melhores candidatos à automação, pois a intervenção humana nessas tarefas representa uma fonte recorrente de erros operacionais e ineficiência. A validação da solução foi conduzida por comparação entre execuções robóticas e humanas, demonstrando ganhos de eficiência com a substituição do processo manual.

O presente trabalho compartilha dessa premissa: a montagem manual do barema é exatamente o tipo de tarefa repetitiva e determinística que se beneficia de automação. A diferença reside na abordagem técnica adotada — enquanto Fernandes automatiza a interação com sistemas legados existentes via RPA, este trabalho desenvolve um sistema original que codifica as regras de negócio do COLCIC diretamente em sua lógica, integrando manipulação de documentos, validação de dados e geração do arquivo final em um único fluxo.

5.2 Sistemas Web para Gerenciamento de Atividades Complementares

Barbosa 2020 desenvolveu, no Instituto de Ciências Exatas e Tecnologia da Universidade Federal do Amazonas, o SCAC — Sistema de Cadastramento das Atividades Complementares — para o curso de Sistemas de Informação. O trabalho parte do diagnóstico de que a comissão de avaliação do curso utilizava uma planilha para controle das horas complementares dos discentes, o que limitava a efetividade do processo. O sistema desenvolvido gerencia as solicitações de atividades complementares e foi avaliado positivamente em testes de usabilidade e funcionalidade junto aos usuários do curso.

Alcântara et al. 2021, em perspectiva complementar, propuseram, em artigo publi-

cado na Revista de Gestão e Avaliação Educacional, a criação de um sistema de processo eletrônico para o aproveitamento de atividades complementares em uma IES brasileira. O estudo, conduzido por meio de observação participante e análise documental, identificou que a transição do processo físico para o eletrônico produz melhora significativa na gestão institucional, reduzindo o retrabalho administrativo e acelerando o fluxo de aproveitamento das horas pelos discentes.

Ambos os trabalhos evidenciam que o problema da gestão manual de atividades complementares é recorrente em diferentes instituições brasileiras e que a automação desse processo gera impacto positivo tanto para discentes quanto para a coordenação. O sistema proposto neste TCC diferencia-se dessas soluções ao incorporar dois elementos não enfatizados nos trabalhos citados: a geração automatizada do documento final unificado com indexação de páginas — o barema — e a pré-validação dos comprovantes submetidos, confrontando as declarações do discente com os dados extraídos dos certificados antes da submissão oficial ao colegiado.

5.3 Extração Automatizada de Dados em Documentos PDF

Wiechork 2021 investigou, em dissertação de mestrado pela Universidade Federal de Santa Maria, abordagens para a extração automatizada de dados de documentos PDF nativamente digitais, aplicando-as a 343 provas do ENADE entre 2004 e 2019. O trabalho avalia ferramentas de extração em três categorias — dados tabulares, conteúdo textual e regiões de interesse — e aponta limitações relevantes relacionadas à diversidade de layouts dos documentos, especialmente quando o conteúdo está disposto em múltiplas colunas ou quando há variações estruturais entre edições do mesmo documento.

Essa limitação é diretamente aplicável ao contexto deste trabalho. Os certificados de atividades complementares submetidos pelos discentes são emitidos por instituições diversas, sem padronização de layout, o que impõe ao sistema de extração o desafio de identificar informações relevantes — como carga horária, nome do beneficiário e data de realização — em documentos estruturalmente heterogêneos. Diferentemente do trabalho de Wiechork, que avalia ferramentas existentes sobre documentos com estrutura relativamente previsível, o sistema proposto neste TCC opera sobre documentos heterogêneos e pouco padronizados, integrando os dados extraídos a um fluxo de pré-validação com regras de negócio específicas do COLCIC.

5.4 Síntese Comparativa

A Tabela 1 sintetiza os principais pontos de aproximação e diferenciação entre os trabalhos analisados e o sistema proposto neste TCC.

Tabela 1 – Comparação entre trabalhos correlatos e o sistema proposto

Trabalho	Foco principal	Relação com este TCC
Fernandes 2021	Automação de processos administrativos universitários por meio de RPA.	Aproxima-se pela automação de tarefas repetitivas, mas difere por automatizar sistemas legados, enquanto este trabalho desenvolve uma aplicação própria para o fluxo do Barema.
Barbosa 2020	Sistema para cadastramento e gerenciamento de atividades complementares.	Aproxima-se pelo domínio das atividades complementares, mas não enfatiza a geração de um PDF final indexado nem a pré-validação automática dos comprovantes.
Alcântara et al. 2021	Processo eletrônico para aproveitamento de atividades complementares em uma instituição de ensino superior.	Aproxima-se pela digitalização do fluxo institucional, mas trata o processo em nível administrativo mais amplo, sem foco específico na montagem automatizada do Barema do COLCIC.
Wiechork 2021	Extração automatizada de dados em documentos PDF.	Aproxima-se pela extração de informações em PDFs, mas este TCC aplica essa extração a certificados heterogêneos e a integra a regras de negócio acadêmicas.

Fonte: Elaborado pela autora (2026).

A comparação evidencia que os trabalhos correlatos abordam dimensões relevantes do problema, como automação administrativa, gestão de atividades complementares e extração de dados em documentos PDF. Entretanto, essas dimensões aparecem de forma separada nos estudos analisados. O sistema proposto neste TCC diferencia-se por integrá-las em um único fluxo voltado ao contexto específico do COLCIC/UESC, reunindo a organização dos comprovantes, a aplicação de regras regulamentares, a pré-validação dos dados declarados e a geração de um documento final unificado e indexado.

Dessa forma, a contribuição do trabalho não está na criação de um novo método geral de avaliação acadêmica, mas na aplicação integrada de técnicas de Engenharia de Software a um problema institucional concreto, com regras próprias, documentos heterogêneos e impacto direto sobre discentes e coordenação.

6 Metodologia

Este capítulo descreve o caminho adotado para conduzir o trabalho e para avaliar se a solução proposta atende ao problema identificado. Para isso, o texto apresenta inicialmente a natureza do projeto necessária para compreensão do problema, seguida pelos procedimentos de desenvolvimento e, por fim, a estratégia de avaliação do protótipo.

6.1 Caracterização do Trabalho

O trabalho caracteriza-se como uma atividade de natureza tecnológica, pois tem como objetivo produzir uma solução computacional para apoiar um processo acadêmico real: a organização e a pré-validação do barema de atividades complementares do Colegiado de Ciência da Computação. O projeto possui abordagem qualitativa, uma vez que parte da análise do fluxo utilizado pelos discentes, das regras institucionais e das dificuldades observadas na preparação dos documentos.

O foco do estudo não é propor um novo método teórico de avaliação acadêmica, mas transformar regras já existentes em uma ferramenta funcional. Assim, o mérito do trabalho está na construção, organização e validação de um protótipo capaz de reduzir erros recorrentes no preenchimento, anexação e conferência inicial dos certificados.

6.2 Procedimentos Metodológicos

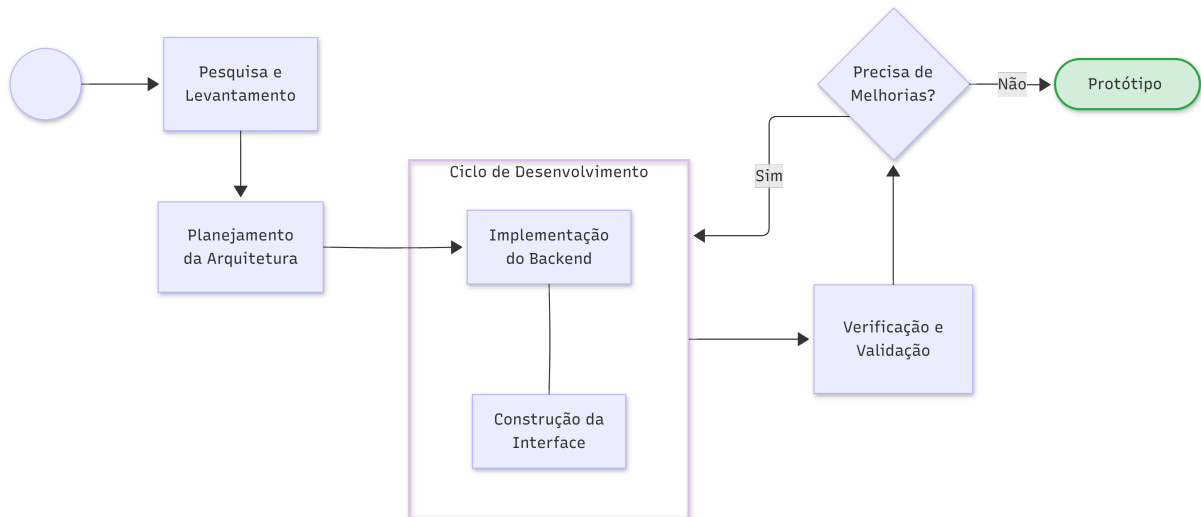
Esta seção detalha as etapas operacionais e metodológicas adotadas para a concepção e construção do sistema. Para estruturar o processo de desenvolvimento de forma lógica e rastreável, o texto apresenta inicialmente o modelo de ciclo de vida escolhido para guiar a engenharia do *software*. Na sequência, descrevem-se os passos práticos realizados, que englobam desde o levantamento técnico das regras institucionais até as diretrizes de arquitetura e codificação da ferramenta.

6.2.1 Prototipagem Evolutiva

O desenvolvimento foi conduzido por prototipagem evolutiva, em que versões sucessivas do sistema foram refinadas a partir das necessidades identificadas durante o levantamento e a implementação (Soares e Resende 2007). Essa estratégia foi adotada porque o problema envolve interação com usuário, regras de negócio específicas e tratamento de documentos em formato PDF, aspectos que exigem ajustes progressivos ao longo do desenvolvimento.

A Figura 1 sintetiza o fluxo adotado durante a construção do protótipo, evidenciando o caráter iterativo do processo. A partir do levantamento inicial de regras para o sistema, foram planejadas e implementadas versões sucessivas da ferramenta, submetidas a verificações e ajustes até que o sistema atendesse às funcionalidades previstas.

Figura 1 – Fluxo de desenvolvimento adotado na construção do protótipo.



Fonte: elaboração própria.

6.2.2 Levantamento de Requisitos

Inicialmente, foi realizado o levantamento das regras do processo de elaboração e envio do barema para elencar as funcionalidades necessárias para o sistema, compreendendo os critérios institucionais que a ferramenta precisaria respeitar. Para isso, foram conduzidas consultas junto à coordenação do COLCIC, com o objetivo de alinhar o fluxo da aplicação aos processos internos do colegiado e compreender como o sistema poderia otimizar o trabalho da coordenação. Além disso, egressos do curso foram consultados a fim de identificar pontos de atenção, dificuldades frequentes e recursos essenciais que não poderiam faltar na plataforma. A partir desse levantamento, definiram-se as principais funcionalidades do sistema, que foram formalizadas com requisitos funcionais e não funcionais na seção 7.2.

Para formalizar o entendimento dessas regras de negócio e preparar o design estrutural do sistema, adotou-se a Linguagem de Modelagem Unificada (UML). A modelagem atuou como uma ferramenta metodológica transitória: os requisitos levantados foram mapeados visualmente em Diagramas de Casos de Uso, enquanto o comportamento do estado transiente foi projetado através de Diagramas de Sequência, servindo de base documentada para a fase posterior de codificação.

6.2.3 Definição da Arquitetura, Ferramentas e Implementação

Após o levantamento, delimitou-se a estrutura macroscópica do sistema através de um modelo arquitetural em camadas baseado em *MVC*, com o propósito de isolar as responsabilidades de apresentação, controle, lógica de negócio e persistência de dados. Essa etapa de *design* estrutural foi fundamental para subsidiar a fase subsequente de desenvolvimento, garantindo que a implementação dos módulos operacionais seguisse um padrão previsível e de fácil manutenção.

Em consonância com a arquitetura definida, procedeu-se à seleção das ferramentas e do ambiente tecnológico mais adequados para suportar os requisitos do projeto. Como critério de escolha, priorizou-se um ecossistema capaz de lidar com processamento assíncrono e manipulação ágil de binários na memória. A partir dessa premissa, definiu-se a utilização da linguagem Python com o microframework Flask para a orquestração do *backend*, a biblioteca *PyMuPDF*TM para a leitura e extração de metadados dos certificados, e ferramentas *web* reativas para a construção da *interface*. Para a eventual gravação de dados, optou-se pelo banco de dados relacional SQLite, operado por meio do ORM SQLAlchemy.

A partir dos diagramas UML estruturados, delineamento arquitetural e seleção de ferramentas, iniciou-se a etapa de implementação propriamente dita (a codificação da aplicação). Nela, os componentes da arquitetura ganharam forma por meio do desenvolvimento dos mecanismos de preenchimento do formulário, da lógica de anexação de certificados, das rotinas de extração de dados e pré-validação dessas informações, culminando na lógica de montagem automatizada do arquivo final. Durante todo esse processo, as funcionalidades foram testadas de forma incremental. A cada novo recurso incorporado, foram verificados os impactos sobre o fluxo completo de uso, assegurando a convergência entre as decisões de código e os objetivos de automação estabelecidos para a plataforma.

6.3 Estratégia de Avaliação

A avaliação do protótipo foi planejada com base em testes funcionais e na verificação rigorosa de conformidade com as regras do Barema. O objetivo dessa etapa é garantir que o sistema processe corretamente as informações de entrada e aplique as validações institucionais de forma precisa, simulando o uso real por parte dos discentes e mitigando erros formais antes da submissão oficial ao colegiado.

O planejamento dos testes de conformidade consistiu no mapeamento das restrições do edital do COLCIC em matrizes de rastreabilidade. A partir dessas matrizes, estabeleceu-se que cada fluxo alternativo ou de erro desenhado para o orquestrador deveria ser provocado intencionalmente por meio de dados de entrada controlados, permitindo avaliar o comportamento adaptativo do *backend* tanto em condições ideais quanto em cenários de

falha.

Para isso, a validação foi estruturada em torno de cenários específicos de uso, que cobrem as principais funcionalidades da ferramenta. Os cenários observados incluem:

- **Anexação de múltiplos certificados:** Análise de como a ferramenta gerencia, armazena temporariamente na memória e processa lotes de arquivos enviados simultaneamente.
- **Identificação de carga horária e metadados:** Teste da capacidade do motor de busca textual em extrair e contabilizar as horas contidas nos documentos binários.
- **Comparação de horas e aplicação de tetos:** Verificação do cálculo entre as horas declaradas pelo discente e o limite máximo permitido por categoria no regulamento.
- **Deteção de certificados duplicados:** Validação da regra de negócio que impede a submissão repetida do mesmo comprovante por meio da comparação do conteúdo bruto.
- **Mecanismo de contingência da pré-validação:** Avaliação da resiliência do sistema e ativação do comportamento seguro básico quando há indisponibilidade de serviços externos de inteligência artificial ou envio de arquivos sem texto extraível.
- **Geração do documento final:** Avaliação da legibilidade, paginação, inserção de separadores e integridade dos dados no PDF consolidado pela aplicação.

6.3.1 Critérios de Sucesso do Protótipo

Para determinar se a solução computacional atende satisfatoriamente ao problema identificado e valida a viabilidade tecnológica da proposta, foram estabelecidos critérios de sucesso objetivos:

- **Eficiência da *Interface*:** As atualizações de estado, substituições de arquivos e sinalizações de erro devem ocorrer de forma assíncrona, mantendo o tempo de resposta ágil e a fluidez da navegação sem recarregamento de página.
- **Precisão na Extração de Metadados:** O motor de busca baseado em *PyMuPDF* deve apresentar eficácia na leitura de certificados com camada de texto padrão, extraíndo corretamente a paginação e identificando os valores de carga horária.
- **Conformidade com o Regulamento:** O sistema deve aplicar rigorosamente as restrições do COLCIC, bloqueando o cômputo de horas excedentes e gerando alertas reativos ao discente em caso de discrepâncias de nome, horas ou temporalidade.

- **Integridade da Unificação:** O arquivo PDF final gerado deve consolidar todos os anexos válidos em um único documento perfeitamente paginado, ordenado e com folhas de rosto divisórias, sem perda de qualidade visual dos binários originais.

Dessa forma, a avaliação busca responder se o protótipo reduz erros antes da submissão oficial, mantém aderência às regras do colegiado e produz um documento final devidamente organizado para a análise da coordenação.

7 Solução Proposta

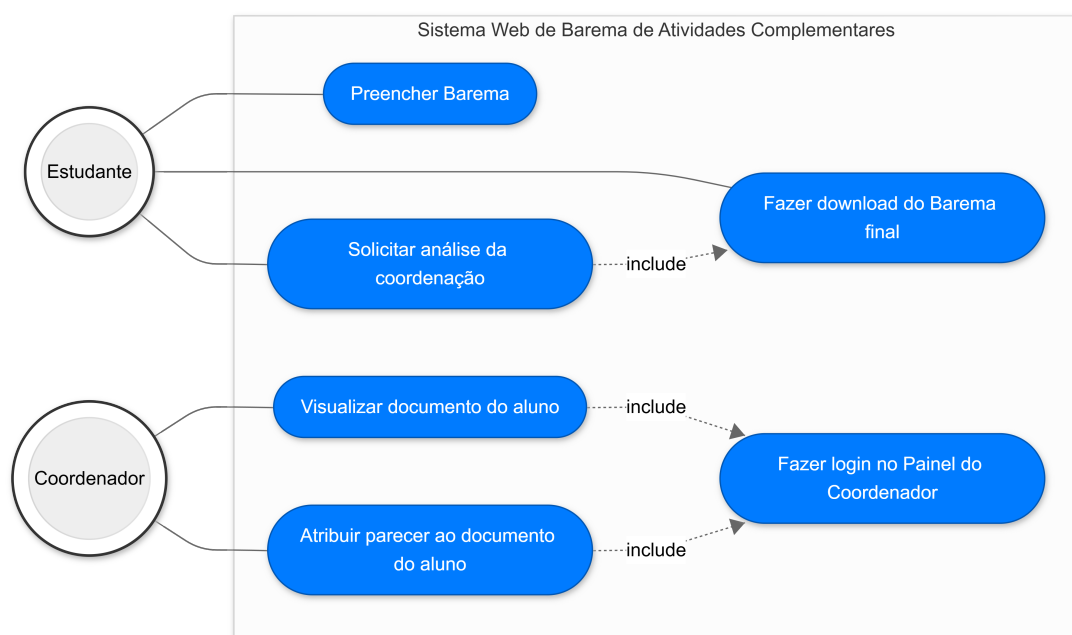
Este capítulo apresenta a solução proposta para apoiar os discentes na organização do barema de atividades complementares conforme discutido na Seção 2.2. A solução consiste em um sistema web que centraliza o preenchimento dos dados do aluno, a anexação dos certificados, a pré-validação das informações e a geração de um documento final em PDF.

7.1 Visão Geral e Casos de Uso

O sistema foi concebido para atuar em uma fase antecedente à submissão oficial do barema (descrita em 2.1), funcionando como uma camada de apoio à organização e conferência inicial dos documentos. O seu objetivo não é substituir a análise de mérito realizada pelo colegiado, mas sim atuar na mitigação ativa de erros comuns no momento do envio, como o cômputo de carga horária incompatível, a submissão de certificados repetidos e a desorganização da paginação dos anexos (melhor descritos na Seção 2.2).

Para mapear as interações entre os usuários e o sistema, bem como definir o escopo das funcionalidades, elaborou-se o Diagrama de Casos de Uso (Figura 2). O diagrama identifica os dois atores principais do sistema — o Discente e o Coordenador — e as ações que cada um pode realizar na plataforma.

Figura 2 – Diagrama de Casos de Uso do sistema Barema.



Fonte: Elaboração própria (2026).

A partir desta modelagem de interações, a arquitetura e a implementação do sistema foram divididas em quatro módulos operacionais lógicos: Montagem do Barema, Extração de Dados, Pré-validação, Geração do Documento e o Painel de Análise dos Baremas.

7.2 Requisitos do Sistema

Conforme delineado no capítulo 6, o levantamento de requisitos que guiou a construção dos módulos operacionais foi formalizado a seguir:

Tabela 2 – Requisitos funcionais do sistema

Código	Descrição
RF01	O sistema deve permitir que o discente anexe, categorize e altere a ordem dos certificados PDF.
RF02	O sistema deve extrair automaticamente a quantidade de páginas dos arquivos PDF enviados.
RF03	O sistema deve comparar a carga horária informada com os tetos máximos permitidos no regulamento.
RF04	O sistema deve emitir alertas visuais ao usuário caso haja divergência entre as regras e os dados inseridos.
RF05	O sistema deve consolidar os arquivos em um único PDF, com indexação e paginação automática.
RF06	O sistema deve calcular e exibir a carga horária total somada dos certificados no documento final.
RF07	O sistema deve verificar se a data de emissão ou realização do certificado está dentro do período ativo de graduação do aluno.
RF08	O sistema deve comparar se a carga horária e o nome informados no formulário coincidem com os dados extraídos do PDF.
RF09	O sistema deve detectar e alertar sobre a submissão de arquivos PDF duplicados.
RF10	O sistema deve permitir que o discente solicite a análise formal do Barema após a geração do documento.
RF11	O sistema deve fornecer um painel administrativo protegido para o coordenador visualizar e gerenciar o histórico de solicitações.
RF12	O sistema deve enviar o <i>feedback</i> da análise automaticamente para o e-mail ou WhatsApp do discente.

Fonte: Elaborado pelo autor (2026).

Também foram detectados os Requisitos Não-Funcionais do Sistema, listados a seguir:

Tabela 3 – Requisitos não funcionais do sistema

Código	Descrição
RNF01	A interface gráfica deve refletir o reposicionamento e a reordenação de arquivos de forma imediata.
RNF02	O processamento das regras e aplicação de tetos deve possuir tempo de resposta ágil, sem travamentos na interface.
RNF03	O painel de gerenciamento administrativo deve ser restrito e protegido por controle de acesso (autenticação).

Fonte: Elaborado pelo autor (2026).

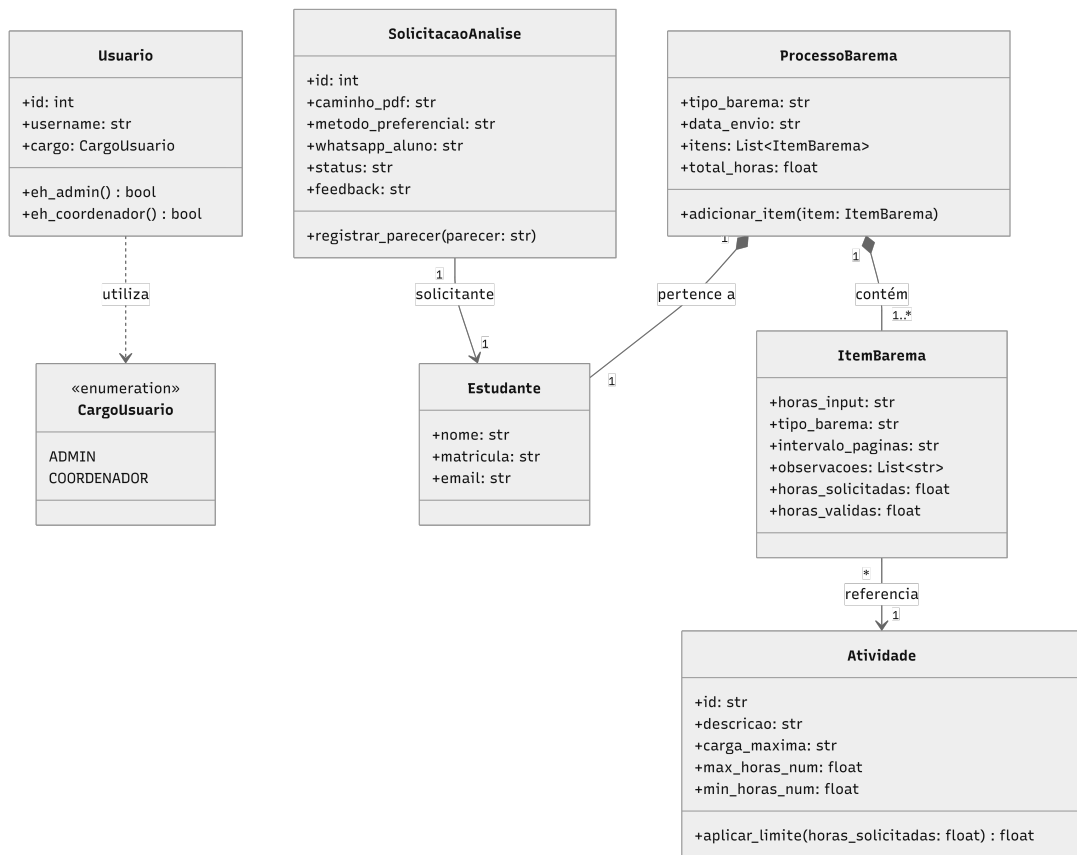
7.3 Arquitetura e Persistência de Dados no Projeto

A aplicação foi estruturada seguindo o padrão de arquitetura em camadas para isolar estritamente as responsabilidades de interação, controle e persistência. No *backend*, a orquestração das rotas e serviços foi implementada utilizando a linguagem Python™ em conjunto com o *microframework* Flask™. A escolha por um *microframework* justifica-se pela necessidade do projeto em gerenciar estados voláteis na memória de forma ágil durante o rascunho do barema, sem a sobrecarga de ferramentas monolíticas.

Para refletir essa separação de responsabilidades no código-fonte, a arquitetura foi inspirada no padrão *Model-View-Controller* (MVC) aliado a princípios de isolamento de domínio. A base de código foi dividida em diretórios lógicos estruturais: o núcleo de regras de negócio foi isolado em `core/entities` (representando as estruturas de dados puras, desacopladas do banco) e `core/services` (que orquestra a lógica de extração e validação do Barema). Já a persistência e a comunicação HTTP foram delegadas a camadas externas, como os *controllers* de rotas e o `models.py`. Esse desacoplamento garante que a lógica de aplicação dos limites de horas não dependa do *framework web* ou do banco de dados.

Para guiar a implementação desse núcleo de negócio e documentar as entidades centrais, elaborou-se o Diagrama de Classes (Figura 3). A modelagem descreve as classes puras do sistema (como a Atividade, o Item do Barema e o Histórico de Solicitações), mapeando seus atributos, métodos de obtenção de dados e as funções responsáveis pela aplicação dos tetos regulamentares.

Figura 3 – Diagrama de Classes do núcleo de domínio do sistema.

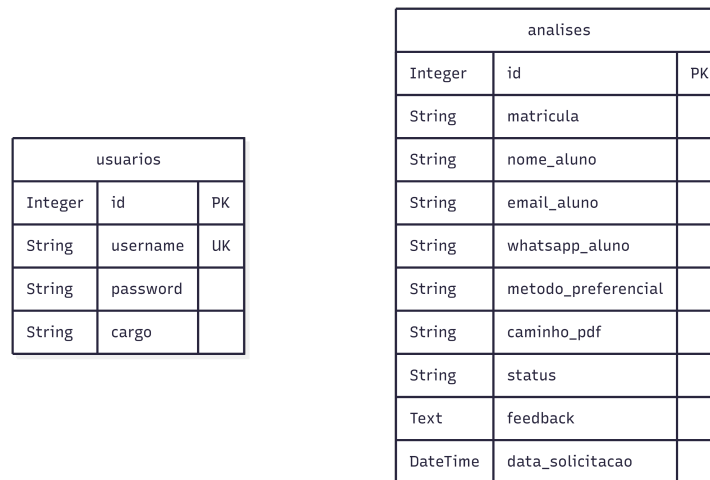


Fonte: Elaboração própria (2026).

O estado persistente do sistema — isto é, o armazenamento definitivo do histórico de solicitações e do vínculo com os arquivos gerados — foi materializado fisicamente através do banco de dados relacional *SQLite*. A comunicação entre os objetos de domínio em Python e as tabelas do banco de dados é mediada pelo *framework Flask-SQLAlchemy*. A adoção deste mapeador objeto-relacional (ORM) permitiu criar métodos de conversão bidirecional (padrão *Data Mapper*), de modo que as entidades fossem salvas no banco de dados abstraindo a complexidade de consultas SQL literais e assegurando transações atômicas seguras no registro dos protocolos submetidos.

O reflexo físico dessa persistência pode ser visualizado no Modelo Entidade-Relacionamento (MER) do banco de dados (Figura 4), que ilustra as tabelas geradas de forma automatizada pelo *SQLAlchemy* para suportar as análises dos discentes e as credenciais dos usuários administrativos.

Figura 4 – Modelo Entidade-Relacionamento (MER) do banco de dados.



Fonte: Elaboração própria (2026).

Por fim, ao observar o modelo estrutural do banco de dados, cabe destacar uma decisão importante sobre a arquitetura do projeto: o sistema não exige *login* dos discentes. Essa escolha foi feita para facilitar o uso, permitindo que o aluno gere seu protocolo de forma direta e sem burocracia. Por conta disso, as informações de identificação (nome, matrícula e *e-mail*) não ficam em uma tabela separada de usuários. Em vez disso, são salvas diretamente no registro da tabela de **analises**. Essa estratégia simplifica a infraestrutura, pois elimina a necessidade de gerenciar o cadastro e a recuperação de senhas dos alunos, restringindo a autenticação de segurança estritamente ao painel da coordenação.

7.4 Implementação

A partir do escopo definido pelos Casos de Uso, a construção do *software* foi segmentada em módulos operacionais. A seguir, detalha-se o funcionamento e as bibliotecas empregadas em cada um destes núcleos.

7.4.1 Montagem do Barema

Este módulo é responsável pela interação primária com o discente, englobando a coleta de dados e a recepção dos arquivos comprobatórios. Inicialmente, o usuário acessa uma tela com um formulário para inserção dos dados básicos de identificação. A matrícula inserida atua como chave de busca para que a lógica de domínio carregue dinamicamente o edital regulamentar adequado (Figura 5).

Figura 5 – Tela inicial do sistema Barema.

Barema de Atividades

Digite sua matrícula para carregar o formulário compatível com seu regulamento.

Monte seu Barema

Acesso administrativo

Nome do Discente:

Matrícula:

Email:

Data de Verificação:

Fonte: Elaboração própria (2026).

Após o preenchimento, a interface renderiza a tabela de atividades. A construção visual e interativa deste módulo foi realizada utilizando *JavaScript* nativo integrado à *Fetch API*. Essa tecnologia foi fundamental para atender ao requisito de usabilidade (RNF01), pois permite a comunicação assíncrona com o *backend*. Dessa forma, o aluno pode anexar PDFs e visualizar o preenchimento da carga horária sem que ocorra o recarregamento da página (Figura 6).

Figura 6 – Formulário do barema com atividades carregadas.

ATIVIDADE	C.H. MÁXIMA	C.H. CUMPRIDA	ANEXAR COMPROVANTE(S)
1. Participação no CACIC: coordenador (a) geral; membro de comissão de assuntos acadêmicos; membro de comissão de eventos; secretário (a); tesoureiro (a)	Máximo de 50 horas	0	Escolher Arquivo + Adicionar outro
2. Manutenção não remunerada dos laboratórios da computação	Máximo de 50 horas	0	Escolher Arquivo + Adicionar outro
3. Participação em eventos científicos relacionados à computação	Máximo de 80h desde que não seja inferior a 1 dia	0	Escolher Arquivo + Adicionar outro
4. Orientando em projeto de Iniciação Científica	Máximo de 60h/projeto	0	Escolher Arquivo + Adicionar outro
5. Atividades especiais apreciadas e aprovadas pelo COLCIC. Inclui, não exclusivamente, cursos de curta duração, presencial ou on-line, com no mínimo 12h desde que comprovado com certificado que possa ter a sua autenticidade verificada.	Máximo de 50h/ano/atividade	0	Escolher Arquivo + Adicionar outro

Fonte: Elaboração própria (2026).

7.4.2 Extração de Dados

Uma vez que o arquivo é recebido pela camada de rotas, o Módulo de Extração é acionado. Este componente foi otimizado para operar diretamente sobre o fluxo de *bytes* (*stream*) na memória volátil do servidor, dispensando a necessidade de gravação prévia em disco. Para garantir a resiliência da aplicação, o motor de extração foi arquitetado para lidar com dois cenários distintos de arquivos comprobatórios: documentos com camada de texto estruturada e imagens rasterizadas.

Para a inspeção de binários e extração nativa de metadados textuais em arquivos PDF padrão, o projeto empregou a biblioteca *PyMuPDF* (através do módulo `fitz`). O serviço instancia o documento na memória utilizando `fitz.open()` e percorre as páginas invocando métodos de captura direta de *strings* legíveis, o que garante máxima velocidade de processamento.

Contudo, para cenários em que o discente anexa arquivos de imagem (extensões como PNG ou JPEG) ou documentos PDF originados de *scanners* (sem camada de texto extraível), o sistema aciona um mecanismo de contingência baseado em Reconhecimento Óptico de Caracteres (OCR). Utilizando o motor *Tesseract OCR* (integrado via biblioteca `pytesseract`), o fluxo visual é processado e os caracteres contidos na imagem são reconhecidos e convertidos em texto estruturado na memória da aplicação.

Independentemente da via de extração acionada (nativa via *PyMuPDF* ou óptica via *OCR*), a *string* resultante é padronizada e submetida ao módulo nativo de Expressões Regulares do Python (`re`). O algoritmo utiliza a função `re.findall()` configurada com padrões morfológicos para localizar a carga horária declarada no texto, isolando números que estejam semanticamente adjacentes a termos específicos (como "horas" ou "h").

7.4.3 Pré-validação e Geração do Documento

Com os dados extraídos em memória, a aplicação executa a Pré-validação. A validação básica cruza os metadados do arquivo com as regras institucionais carregadas no edital, verificando discrepâncias na carga horária, titularidade e submissões em duplicidade (comparando o conteúdo bruto dos binários).

Além das verificações determinísticas, este módulo foi arquitetado para suportar uma camada opcional de contingência e flexibilidade textuais operada por recursos de inteligência artificial. Se a *API* externa estiver configurada e disponível, o sistema aproxima semanticamente o contexto do certificado com a categoria exigida; em caso de falha de conexão, o sistema adota um comportamento de resiliência e retorna à validação básica.

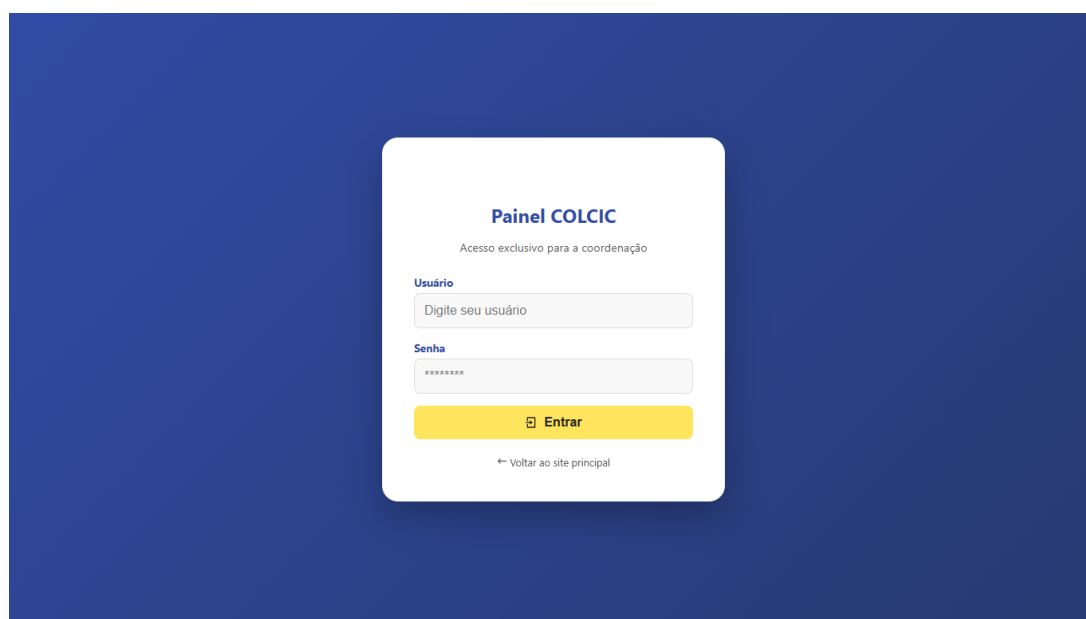
Superadas as validações, o processo culmina na geração do documento. Utilizando novamente os recursos de manipulação do *PyMuPDF*, o *backend* consolida os dados do formulário, cria laudas de indexação separadoras dinamicamente e anexa os comprovantes

validados, exportando um único PDF unificado e devidamente paginado para o aluno.

7.4.4 Painel de Análise (Coordenação)

O último componente do escopo lida com a tramitação oficial do documento gerado. Este módulo possui acesso restrito e requer autenticação, separando o fluxo público discente do acesso gerencial da coordenação (Figura 7).

Figura 7 – Tela de *login* administrativo.

A imagem mostra a tela de login administrativo do Painel COLCIC. O fundo é azul escuro. No centro, há um formulário branco com o título "Painel COLCIC" em azul. Abaixo do título, o texto "Acesso exclusivo para a coordenação" aparece em cinza. O formulário contém dois campos de entrada: "Usuário" com o placeholder "Digite seu usuário" e "Senha" com caracteres ocultos por pontos. Abaixo dos campos, há um botão amarelo com o ícone de uma chave e o texto "Entrar". Na base do formulário, há um link "← Voltar ao site principal" em cinza.

Fonte: Elaboração própria (2026).

Ao confirmar o envio do barema, o Módulo de Análise converte os dados da memória transiente para o estado persistente, criando um registro de protocolo via SQLAlchemy na base SQLite. O coordenador, devidamente autenticado, acessa uma interface gerencial que consome esses registros, permitindo a filtragem de *status* e o acesso direto ao PDF unificado gerado no módulo anterior (Figura 8).

Figura 8 – Painel do coordenador para análise das solicitações.



Fonte: Elaboração própria (2026).

O ciclo de caso de uso deste módulo encerra-se com o registro do parecer institucional no sistema e a consequente notificação do discente. Para garantir a transparência do processo, o sistema implementa um serviço de mensageria acionado por gatilhos de evento (*event triggers*). Assim que o coordenador submete a avaliação e o *status* do protocolo é atualizado no banco de dados (acionando a função `registrar_parecer()`), a aplicação intercepta essa mudança de estado e invoca o módulo de comunicação.

Embasado nos dados persistidos, o sistema verifica o `metodo_preferencial` escolhido pelo discente durante o preenchimento. Para a orquestração e o disparo automático dessas mensagens, optou-se pela integração com a plataforma de comunicações em nuvem *Twilio*. A escolha dessa ferramenta justifica-se pela sua capacidade de centralizar múltiplos canais de notificação (*omnichannel*) em uma única API segura, escalável e de alta disponibilidade.

Caso a escolha do aluno seja por correio eletrônico, o *backend* aciona a API de *email* da plataforma (*Twilio SendGrid*) para despachar o *feedback* detalhado. Se a preferência for pelo aplicativo *WhatsApp*, o orquestrador consome a API de mensageria nativa do *Twilio*, entregando a notificação de forma síncrona e formatada diretamente no dispositivo móvel do discente - também é enviado o feedback por *email* -. Essa automatização conclui o fluxo do projeto, eliminando por completo a carga de trabalho operacional da coordenação com o envio manual de retornos e garantindo que o discente seja informado de imediato sobre o encerramento institucional do seu processo.

8 Resultados e Discussão

Este capítulo apresenta os resultados obtidos com a implementação do protótipo, os cenários detalhados utilizados para sua homologação e uma discussão técnica sobre o papel da solução na preparação automatizada do Barema do COLCIC. A estratégia de validação adotada neste trabalho segue o planejamento previsto e os parâmetros definidos na Seção 6.3.1, confrontando as saídas geradas pelo sistema com os critérios de sucesso estabelecidos para a aplicação. Todos os resultados descritos concentram-se em evidências funcionais mensuradas em ambiente de testes local.

8.1 Resultados Obtidos

O principal resultado prático deste trabalho consiste na consolidação de uma plataforma *web* funcional projetada especificamente para mediar a preparação de documentos acadêmicos regulatórios. O sistema foi estruturado como um ambiente transiente de trabalho, permitindo que o discente consolide seus dados de matrícula, selecione o barema de transição aplicável, informe as horas declaradas e anexe os comprovantes correspondentes de forma assistida.

Para gerenciar as interações com os arquivos sem sobrecarregar o servidor, o sistema utiliza uma *interface* baseada em componentes reativos em *JavaScript* que se comunicam de maneira assíncrona com a camada de serviços do FlaskTM. O usuário realiza a anexação de novos certificados através de campos de *upload* dinâmicos; o sistema intercepta o arquivo e o envia via *Fetch API* para o *backend*, onde o orquestrador o aloca em uma lista de objetos temporária na memória da sessão. Para substituir ou remover documentos, o usuário conta com ícone e botões intuitivos na *interface*. A substituição e a remoção de documentos ocorrem por meio da manipulação direta dos índices dessa lista volátil, garantindo respostas visuais imediatas na interface sem interrupções ou necessidade de recarregamento total da página.

Outro resultado relevante foi a estruturação do mecanismo de inspeção técnica de binários operado pela biblioteca PyMuPDFTM. No instante em que um certificado é posicionado em uma categoria, o servidor abre o fluxo de *bytes* do arquivo diretamente na memória e executa rotinas baseadas em expressões regulares para identificar padrões textuais de carga horária e datas de realização. Esse processamento imediato alimenta o motor de regras da aplicação, permitindo que o sistema atualize os contadores visuais e aplique os tetos regulamentares na *interface* de maneira instantânea.

A integração entre a extração e a pré-validação de dados demonstrou utilidade

prática ao atuar como um filtro de qualidade anterior à submissão oficial. Ao cruzar as informações textuais do PDF com as declarações do estudante, a plataforma sinaliza inconsistências operacionais, tais como divergências entre o nome informado e o nome identificado no certificado, declaração de carga horária superior à carga textual encontrada no documento e possível duplicidade de certificados. Dessa forma, o sistema não substitui a análise do colegiado, mas antecipa pontos de atenção que poderiam gerar retrabalho durante a conferência institucional.

8.2 Validação do Protótipo

A validação do protótipo foi conduzida por meio de testes de conformidade normativa, simulando cenários reais enfrentados pela coordenação do curso. A estrita aderência às regras do Barema é mandatória, visto que o documento final possui valor regulatório e subsidia o deferimento da colação de grau pelo Colegiado; qualquer erro de cálculo ou categorização inadequada resulta no indeferimento do processo e em prejuízos ao cronograma acadêmico do discente. Para cobrir essas variáveis, foram projetados cenários de testes funcionais, explicados a seguir.

Nos cenários avaliados, o sistema conseguiu identificar inconsistências objetivas antes da submissão oficial. A divergência de nome foi detectada pela comparação entre o nome informado pelo discente e o nome identificado no certificado. A carga horária solicitada foi comparada com o total extraído dos documentos anexados, permitindo apontar quando o valor declarado era superior ao comprovado. A duplicidade foi verificada pela comparação do conteúdo textual dos certificados.

Também foi testado o comportamento da pré-validação quando há indisponibilidade de serviços externos de inteligência artificial. Nesse caso, o sistema retornou para a validação básica, mantendo a análise de regras objetivas sem depender exclusivamente de uma *API* externa. Esse comportamento é relevante porque preserva a utilidade do sistema mesmo quando recursos adicionais de análise textual não estão disponíveis.

Além das validações individuais, foi verificada a geração do documento final. O PDF consolidado reuniu os dados preenchidos e os certificados anexados em uma estrutura única, adequada para encaminhamento à análise oficial. Esse resultado atende à necessidade de reduzir erros de montagem e de organização dos anexos.

8.3 Discussão da Solução

Os resultados indicam que a solução proposta se diferencia de ferramentas genéricas de manipulação de PDF por incorporar regras específicas do barema do COLCIC. Enquanto ferramentas externas apenas permitem juntar ou reorganizar arquivos, o sis-

tema desenvolvido combina a montagem do documento com verificações relacionadas ao regulamento e ao conteúdo dos certificados.

A arquitetura em camadas adotada conferiu flexibilidade e isolamento de escopo ao projeto. A separação clara entre a camada de *interface*, as rotas do Flask™, as funções de serviço do orquestrador e os *drivers* de geração de documentos permitiu que os limites normativos fossem codificados diretamente no *backend* através de um dicionário de restrições explícito. Essa organização facilitou a escrita de rotinas de testes unitários automatizados, validando isoladamente os módulos como cálculo dos pisos e tetos das atividades, extração de carga horária e pré-validação.

Apesar dos resultados promissores, a plataforma deve ser interpretada como um instrumento de apoio à decisão e conferência preliminar, não substituindo o papel avaliador da coordenação do curso. O sistema reduz os erros formais e inconsistências matemáticas evidentes antes do envio, mas a prerrogativa pedagógica de deferimento das atividades e análise de mérito dos certificados permanece estritamente institucional.

8.4 Limitações Observadas

Antes de delimitar as restrições do cenário atual, cabe destacar a robustez técnica demonstrada pelo protótipo no processamento estável de fluxos assíncronos e na fidelidade do documento consolidado gerado. Contudo, foram mapeadas limitações operacionais inerentes à tecnologia empregada que devem ser documentadas.

A primeira limitação reside na dependência da qualidade estrutural do arquivo PDF enviado. A extração automática de dados textuais via PyMuPDF™ apresenta melhor desempenho quando o documento possui uma camada de texto acessível; certificados compostos apenas por imagens ou escaneamentos de baixa resolução podem demandar o uso de Reconhecimento Óptico de Caracteres (OCR). Embora o projeto preveja suporte a esse tipo de recurso como mecanismo complementar, sua efetividade depende da qualidade do arquivo, da configuração do ambiente e da capacidade de reconhecimento do motor utilizado. Ademais, as rotinas que utilizam chamadas de inteligência artificial externa ficam condicionadas à latência de rede e à estabilidade do provedor escolhido, reforçando a importância do mecanismo de contingência estruturado no *backend*.

Por fim, destaca-se que todas as homologações foram realizadas em ambiente local controlado. Embora os cenários simulados reflitam com precisão as regras do COLCIC, a ausência de testes de usabilidade com uma amostra estatisticamente heterogênea de discentes e coordenadores impede a coleta de métricas empíricas sobre a redução real de tempo administrativo e a curva de aprendizado da *interface* em produção, configurando uma oportunidade para trabalhos futuros.

9 Conclusão

Este trabalho apresentou a concepção, implementação e avaliação de um protótipo de sistema *web* voltado ao apoio da preparação do Barema de Atividades Complementares do curso de Ciência da Computação da UESC. A proposta partiu de um problema institucional concreto: a elaboração manual do Barema exigia que o discente organizasse certificados, calculasse cargas horárias, aplicasse limites regulamentares e indexasse páginas de forma dispersa e suscetível a erros. Como consequência, o processo demandava retrabalho tanto dos estudantes quanto da coordenação.

A principal contribuição deste trabalho foi transformar essa etapa manual e fragmentada em um fluxo computacional assistido. Com o protótipo desenvolvido, o discente passa a preencher os dados, anexar certificados, acompanhar alertas de inconsistência e gerar um documento final unificado em uma mesma plataforma. Dessa forma, tarefas antes realizadas de forma separada, como organização dos comprovantes, cálculo de horas, aplicação de tetos e montagem do PDF, passam a ser apoiadas por regras de negócio implementadas no sistema.

Em relação aos objetivos estabelecidos, o trabalho realizou o levantamento das necessidades associadas ao processo de preparação do Barema, estruturou uma aplicação *web* para organização dos dados e certificados, implementou mecanismos de extração e pré-validação de informações, aplicou regras regulamentares de carga horária e gerou um arquivo PDF consolidado, indexado e adequado ao encaminhamento para análise. Também foi estruturado um fluxo de persistência das solicitações, permitindo apoiar o acompanhamento administrativo pela coordenação.

Os resultados obtidos indicam que o protótipo contribui para reduzir falhas formais recorrentes no processo de submissão do Barema. Ao confrontar informações declaradas pelo discente com dados extraídos dos documentos anexados, o sistema permite identificar inconsistências antes da submissão oficial, como divergências de nome, cargas horárias incompatíveis e duplicidade de comprovantes. Além disso, a geração automatizada do documento final reduz a dependência da paginação manual, um dos pontos mais sensíveis do processo anterior.

Do ponto de vista técnico, a solução demonstrou a viabilidade de integrar, em uma única aplicação, manipulação de arquivos PDF, extração textual, validação de regras de negócio, interface reativa e persistência de solicitações. Essa integração representa a contribuição central do trabalho: não se trata apenas de unir arquivos PDF, mas de aplicar conhecimento de Engenharia de Software a um fluxo acadêmico regulado por regras específicas do COLCIC.

Apesar dos resultados alcançados, o sistema deve ser compreendido como um protótipo de apoio à preparação e conferência preliminar do Barema, não como substituto da análise institucional realizada pelo colegiado. A decisão final sobre o aproveitamento das atividades permanece sob responsabilidade da coordenação, especialmente nos casos que envolvem interpretação de mérito acadêmico dos certificados apresentados.

Algumas limitações também devem ser consideradas. A validação foi realizada em ambiente local controlado, sem implantação em produção e sem testes sistemáticos de usabilidade com uma amostra ampla de discentes e coordenadores. Além disso, a extração automática de informações depende da qualidade estrutural dos documentos enviados, podendo apresentar restrições em certificados digitalizados com baixa legibilidade ou formatos pouco padronizados.

Como trabalhos futuros, sugere-se ampliar os testes de usabilidade com usuários reais, avaliar o desempenho da aplicação em ambiente de produção, integrar o fluxo com sistemas institucionais de autenticação e protocolo acadêmico e expandir o painel administrativo com relatórios de acompanhamento. Também pode ser considerada a evolução dos mecanismos de extração e validação, de modo a lidar melhor com documentos heterogêneos e reduzir ainda mais a necessidade de conferência manual.

Em síntese, o trabalho demonstrou que é viável apoiar computacionalmente uma etapa relevante do processo de reconhecimento de Atividades Complementares. O que antes dependia de montagem manual, cálculos isolados e revisão sucessiva de documentos passa a contar com um fluxo integrado de organização, pré-validação e geração automatizada do Barema, preservando o papel avaliador do colegiado e oferecendo uma base concreta para futuras evoluções institucionais.

Referências

ABIDEMI, A. The role of technology and automation in streamlining business processes and productivity for SMEs. *International Journal of Entrepreneurship*, v. 7, n. 3, p. 31, 2024. ISSN 2510-4100. Acesso em: mar. 2026. Disponível em: <https://ajpojournals.org/journals/ije/article/view/2510/4100>. 13, 20

ALCÂNTARA, E. S. d. et al. Processo eletrônico de aproveitamento de atividades complementares: proposta de criação numa IES. *Regae: Revista de Gestão e Avaliação Educacional*, v. 9, n. 18, p. 1–16, 2021. 26, 28

Artifex Software, Inc. *PyMuPDF: Python Bindings for MuPDF*. 2026. Disponível em: <https://pymupdf.readthedocs.io>. Acesso em: 12 abr. 2026. 24

BARBOSA, R. R. *SCAC: Sistema de Cadastramento das Atividades Complementares*. Dissertação (Trabalho de Conclusão de Curso (Bacharelado em Engenharia de Software)) — Universidade Federal do Amazonas, Itacoatiara, AM, 2020. Disponível em: <http://riu.ufam.edu.br/handle/prefix/5834>. 26, 28

BRASIL. *RESOLUÇÃO CNE Nº 2, DE 18 DE JUNHO DE 2007. Dispõe sobre carga horária mínima e procedimentos relativos à integralização e duração dos cursos de graduação, bacharelados, na modalidade presencial*. Brasília, DF: Ministério da Educação, 2007. https://www.gov.br/mec/pt-br/media/acesso_informacao/pdf-arq/rces002_07.pdf. Acesso em: mar. 2026. 13

BRASIL. *RESOLUÇÃO Nº 5, DE 16 DE NOVEMBRO DE 2016. Institui as Diretrizes Curriculares Nacionais para os cursos de graduação na área da Computação*. Brasília, DF: Ministério da Educação, 2016. https://www.gov.br/mec/pt-br/cne/pdf/normas-classificadas-por-assunto/diretrizes-curriculares-cursos-de-graduacao/rces005_16.pdf. Acesso em: mar. 2026. 13

FERNANDES, L. M. *Aplicação da automação robótica de processos em atividades administrativas da Universidade Federal de Santa Catarina*. Dissertação (Trabalho de Conclusão de Curso) — Universidade Federal de Santa Catarina, Araranguá, SC, 2021. Disponível em: <https://repositorio.ufsc.br/handle/123456789/229377>. 26, 28

FOWLER, M. *Padrões de Arquitetura de Aplicações Corporativas*. [S.l.]: Bookman Editora, 2006. 22

Google. *Tesseract Open Source OCR Engine*. 2026. Disponível em: <https://github.com/tesseract-ocr/tesseract>. Acesso em: 12 mar. 2026. 24

GRINBERG, M. *Flask Web Development: Developing Web Applications with Python*. 2. ed. [S.l.]: O'Reilly Media, 2018. 24

HOPCROFT, J. E.; MOTWANI, R.; ULLMAN, J. D. *Introduction to Automata Theory, Languages, and Computation*. 3. ed. Boston: Addison-Wesley, 2006. ISBN 978-0-321-46225-1. 23

- iLovePDF. *Ferramentas de PDF online para amantes de PDF*. 2026. Acessado em: 12 abr. 2026. Disponível em: <<https://www.ilovepdf.com/pt>>. 17
- LAUDON, K. C.; LAUDON, J. P. *Sistemas de Informação Gerenciais*. 11. ed. [S.l.]: Pearson Education, 2014. 17
- NIELSEN, J. *Response Times: The 3 Important Limits*. Nielsen Norman Group, 1993. Acessado em: 04 abr. 2026. Disponível em: <<https://www.nngroup.com/articles/response-times-3-important-limits/>>. 23
- PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de Software: Uma Abordagem Profissional*. 8. ed. Porto Alegre: AMGH, 2016. Disponível em: Internet Archive. Acessado em: 12 mar. 2026. Disponível em: <<https://archive.org/details/pressman-engenharia-de-software-uma-abordagem-profissional-8a>>. 20, 22
- SOARES, B. C.; RESENDE, R. S. F. *Requisitos para utilização de prototipagem evolutiva nos processos de desenvolvimento de software para Web*. Belo Horizonte, MG, 2007. 29
- SQLAlchemy Authors. *SQLAlchemy Documentation*. 2026. Disponível em: <<https://docs.sqlalchemy.org/>>. Acesso em: 12 abr. 2026. 25
- Twilio Inc. *Twilio Documentation and APIs*. 2026. Disponível em: <<https://www.twilio.com/docs>>. Acesso em: 10 abr. 2026. 25
- WIECHORK, K. *Extração automatizada de dados de documentos em formato PDF: aplicação a grandes conjuntos de exames educacionais*. Dissertação (Dissertação (Mestrado em Ciência da Computação)) — Universidade Federal de Santa Maria, Santa Maria, RS, 2021. Disponível em: <<http://repositorio.ufsm.br/handle/1/23130>>. 27, 28